

Session 3 : Introduction des lois de Newton et la méthode Euler

Objectifs de la Session :

- Réviser les sessions précédentes (la cinématique).
- Introduire les lois de Newton.
- Relier les précédentes sessions (la cinématique) aux lois de Newton.
- Introduire la méthode Euler pour résoudre les équations de la cinématique.
- Setup de git, vs code, live preview et Tynymist, pour avoir un accès en ligne et en local à l'ensemble des documents.
- Réaliser un premier travail pratique: Observer une de résolution d'équations cinématiques avec la méthode Euler dans un exemple simple.

Les 3 Lois de Newton

🔗 Physique du Jeu : Les 3 Lois de Newton

Ou "Comment programmer l'univers"

Isaac Newton n'a jamais codé en JavaScript, mais sans lui, aucun jeu vidéo moderne n'existerait. Ses trois lois sont les règles fondamentales du moteur physique que nous allons construire.

Les lois de Newton sont des lois mathématiques qui décrivent comment les objets se déplacent et se comportent sous l'effet des forces. Elles nous permettent de donner un comportement réaliste à nos objets dans le jeu.

En physique, on crée des modèles, en se basant sur nos observations, puis on confronte ces modèles à la réalité, pour valider ou invalider ces modèles. À aucun moment les physiciens prétendent que leurs modèles sont exactes, mais une loi est utile à partir du moment où aucune expérience ne peut prouver qu'elle est fausse.

1. La Première Loi : L'Inertie

Le principe du statu quo

La Loi

Un objet au repos reste au repos, et un objet en mouvement continue en ligne droite à vitesse constante, **sauf** si une force extérieure agit sur lui.

Ce que ça veut dire : Les objets ont une résistance naturelle au changement. Ils veulent continuer à faire ce qu'ils font déjà. Pour changer la vitesse d'un objet (accélérer, freiner, tourner), il **faut** appliquer une force.

Exemple "Vie de tous les jours" : Vous êtes debout dans le bus. Le bus freine brusquement. Votre corps continue d'avancer et vous manquez de tomber. C'est l'inertie : votre corps voulait garder sa vitesse.

Exemple "Jeu Vidéo" :

- **Space Invaders / Mario (Sol) :** Quand vous lâchez la manette, le personnage s'arrête instantanément. *C'est physiquement faux !* (C'est comme s'il y avait des frottements infinis).
- **Asteroids / Dead Space (Espace) :** Vous donnez une impulsion au vaisseau. Même si vous lâchez les commandes, le vaisseau continue d'avancer pour toujours dans la même direction. C'est ça, la vraie inertie (pas de frottements dans l'espace).

- **Niveaux de glace** : Pourquoi Mario glisse ? Parce que les frottements (la force extérieure qui freina) sont réduits. L'inertie reprend le dessus.

2. La Deuxième Loi : La Dynamique

L'équation du Moteur Physique

C'est la loi la plus importante pour nous, car c'est celle qu'on code littéralement dans `animate()`.

La Formule

$\vec{F} = m \cdot \vec{a}$ Dans notre cas, on cherche l'accélération :

$$\vec{a} = \frac{\vec{F}}{m}$$

Ce que ça veut dire : L'accélération ($\vec{a}$) dépend de deux choses :

1. Une force : ($\vec{F}$).
2. La lourdeur intrinsèque de l'objet (m).

Exemple "Vie de tous les jours" : Imaginez un caddie de supermarché.

- **Vide (m petit)** : Une petite force le fait partir à toute vitesse.
- **Plein d'eau (m grand)** : La même force le fait à peine bouger.

Exemple "Jeu Vidéo" (Balancing) : Imaginez un jeu de tir (FPS) avec deux classes de personnages :

- **Le Scout (Masse = 50kg)** : Si le joueur appuie sur "Avancer" (Force = 500N), il accélère à 10m/s^2 . Il est agile.
- **Le Tank (Masse = 200kg)** : Avec la même touche "Avancer" (Force = 500N), il accélère seulement à 2.5m/s^2 . Il est lent et lourd.

Dans Three.js : C'est ici qu'on calcule accélération. Si on veut simuler du vent, de la gravité ou des ressorts, on additionne toutes les forces, on divise par la masse, et on obtient l'accélération pour mettre à jour la vitesse en utilisant la méthode d'Euler. Et de la vitesse, on modifie la position, toujours en utilisant la méthode d'Euler une deuxième fois.

3. La Troisième Loi : Action-Réaction

Les forces sont des paires

La Loi

Pour toute action (force), il y a une réaction égale et opposée.

$$\vec{F}_{A \rightarrow B} = -\vec{F}_{B \rightarrow A}$$

Ce que ça veut dire : On ne peut pas toucher sans être touché. Si vous poussez un mur, le mur vous pousse en retour. Les forces vont toujours par paires.

Exemple "Vie de tous les jours" :

- **Le Skateboard** : Pour avancer, vous poussez le sol vers l'**arrière** avec votre pied. En réaction, le sol pousse votre pied (et vous) vers l'**avant**.
- **Le Ballon de baudruche** : L'air est éjecté vers l'arrière, le ballon part vers l'avant.

Exemple "Jeu Vidéo" :

- **Recul des armes (Recoil)** : Dans *Call of Duty*, quand vous tirez une balle vers l'avant (Action), votre arme et votre caméra remontent vers l'arrière (Réaction).
- **Rocket Jump (Quake / TF2)** : Vous tirez une roquette sur le sol. L'explosion exerce une force vers le bas sur le sol. En réaction, le sol (et l'explosion) exerce une force vers le haut sur le joueur, le propulsant dans les airs.
- **Collisions (Billard)** : Quand la boule blanche tape une boule rouge, la blanche s'arrête (ou recule) car la rouge lui a "rendu" le choc.

Résumé pour le développeur

Loi	Concept	Implémentation Code
1. Inertie	Les objets gardent leur vitesse.	Ne remettez pas <code>velocity</code> à 0 à chaque frame ! Laissez-la vivre entre les frames.
2. $F = ma$	Force $\rightarrow$ Accélération.	<code>acc = forces.divideScalar(mass)</code>
3. Action-Réaction	Les interactions sont doubles.	Collision : Si A repousse B, alors B doit repousser A avec la même force.

Table 1: Aide-mémoire des lois de Newton

Annexe : Pourquoi des objets de masses différentes tombent-ils avec la même vitesse ?

La "force de pesanteur" est la force d'attraction gravitationnelle exercée par un astre (comme la Terre) sur un corps, et qui est aussi appelée "poids". Elle est calculée par la formule :

$$\vec{P}_g = m \cdot \vec{g}$$

La deuxième loi de Newton nous dit que l'accélération est proportionnelle à la force :

$$\vec{a} = \frac{\vec{F}}{m} = \frac{m \cdot \vec{g}}{m} = \vec{g}$$

Donc, l'accélération est la même pour tous les objets, même si la force qui s'applique à eux est proportionnelle à leur masse.

Intégration Numérique & Série de Taylor

💡 Comment l'ordinateur prédit le futur

Dans nos simulations (Three.js), le temps n'est pas continu. L'ordinateur calcule le monde image par image (environ 60 fois par seconde). Nous devons donc transformer les équations physiques continues (dérivées) en instructions discrètes (additions).

1. La Méthode d'Euler

La méthode d'Euler est l'approche la plus intuitive : elle suppose que la vitesse est constante entre deux images.

L'Algorithme (La boucle de jeu)

Pour chaque pas de temps Δt (ex: 0.016s) :

1. **Calculer l'accélération** (Forces / Masse) :

$$\vec{a}_n = \frac{\vec{F}}{m}$$

2. **Mettre à jour la vitesse** (Vitesse actuelle + Changement) :

$$\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \cdot \Delta t$$

3. **Mettre à jour la position** (Position actuelle + Déplacement) :

$$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \cdot \Delta t$$

2. D'où ça vient ? (La Dérivée Discrète)

Mathématiquement, la vitesse est la dérivée de la position. Si on enlève la limite $\lim_{\Delta t \rightarrow 0}$, on obtient une approximation :

$$\vec{v}(t) = \frac{d\vec{r}}{dt} \approx \frac{\vec{r}(t + \Delta t) - \vec{r}(t)}{\Delta t}$$

En isolant le terme futur $\vec{r}(t + \Delta t)$, on retombe sur la formule d'Euler :

$$\underbrace{\vec{r}(t + \Delta t)}_{\text{Futur}} \approx \underbrace{\vec{r}(t)}_{\text{Présent}} + \underbrace{\vec{v}(t) \cdot \Delta t}_{\text{Pas}}$$

3. La Série de Taylor (Le Moteur Mathématique)

La méthode d'Euler n'est en fait que la "pointe de l'iceberg" d'un concept plus puissant : la **Série de Taylor**. Elle stipule que n'importe quelle fonction (trajectoire) peut être reconstruite en additionnant ses dérivées successives.

L'intuition du conducteur aveugle : Si vous fermez les yeux au volant, comment deviner où vous serez dans 1 seconde (Δt) ?

- **Ordre 0 (Position)** : "Je ne bouge pas." $\rightarrow \vec{r}(t)$
- **Ordre 1 (Vitesse)** : "J'avance tout droit." $\rightarrow \vec{v}(t)\Delta t$
- **Ordre 2 (Accélération)** : "J'accélère, donc je courbe." $\rightarrow \frac{1}{2}\vec{a}(t)\Delta t^2$
- **Ordre 3 (A-coup)** : "J'appuie de plus en plus fort..." $\rightarrow \dots$

La Formule Complète :

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}(t)\Delta t + \frac{1}{2}\vec{a}(t)\Delta t^2 + \frac{1}{6}\vec{j}(t)\Delta t^3 + \dots$$

Pourquoi Euler est une approximation ?

La méthode d'Euler **coupe** la série de Taylor après le terme de vitesse.

$$\begin{aligned} \vec{r}_{n+1} &= \vec{r}_n + \vec{v}_n \Delta t \\ &+ \underbrace{O(\Delta t^2)}_{\text{Termes ignorés (Erreur)}} \end{aligned}$$

C'est pour cela qu'on dit qu'Euler est une méthode du **Premier Ordre**. L'erreur commise à chaque pas est proportionnelle au carré du pas de temps (Δt^2).

4. Conséquences Pratiques

Puisque Euler ignore la courbure de la trajectoire (l'accélération) à l'intérieur d'un pas de temps, il a tendance à "déraper" vers l'extérieur des virages ou à gagner de l'énergie (spirale vers l'extérieur) dans des systèmes orbitaux.

Euler (Ordre 1)	Intégration exacte (Ordre 2)
$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t$ <i>Prend la pente au début et trace une ligne droite.</i>	$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t + \frac{1}{2} \vec{a} \Delta t^2$ <i>Prend en compte la courbure (accélération).</i>
Facile à coder. Rapide.	Exact pour la gravité constante (projectiles).

Table 2: Comparaison pour un pas de temps Δt

Conclusion pour le projet : Pour des projectiles simples soumis à une gravité constante, nous pourrions utiliser la formule exacte (Ordre 2). Cependant, Euler reste très utile car il fonctionne même quand l'accélération change tout le temps (ex: vent, ressorts, collisions), là où la formule exacte devient trop complexe.